

LiveLLNN: Reconfigurable, Heterogeneous Lookup Table-Based Logical Neural Networks

Raphael Huang, Marco Frank, Chad Baker

*Baskin School of Engineering, Dept. of Computer Science and Engineering
University of California, Santa Cruz
Santa Cruz, USA*

Abstract—Deploying neural networks on FPGAs typically requires repeated hardware compilation, including synthesis, place-and-route, bitstream generation, and bitstream flashing whenever model parameters change, significantly slowing development and deployment. This work presents LiveLLNN, a reconfigurable lookup-table (LUT) logical neural network accelerator that eliminates this overhead by enabling runtime configuration of LUT-based neurons. Instead of hard-coding trained LUT values into hardware, LiveLLNN implements a configurable inference engine where neural network parameters are updated through reconfigurable LUTs. The architecture is implemented on a heterogeneous Zynq SoC using the PYNQ framework, allowing dynamic model updates via Python and AXI-Lite communication without regenerating the FPGA bitstream. By leveraging configurable FPGA LUT primitives, LiveLLNN enables rapid model iteration while preserving the low latency and efficiency of FPGA-based inference. Experimental results demonstrate flexible deployment of LUT-based neural networks with real-time performance and efficient hardware utilization.

Index Terms—LUT-based neural networks, differentiable logic gates, LUT, lookup tables, FPGAs, heterogeneous computing, neural networks, ZYNQ, PYNQ

I. INTRODUCTION

Recent advances in machine learning (ML) have led to the widespread deployment of neural network models across a variety of applications, including computer vision, natural language processing, and edge intelligence. As model sizes and computational demands continue to grow, a significant effort has been devoted to optimizing neural network architectures with a focus on improving hardware efficiency. Hardware such as GPUs, TPUs, and other accelerators focus on improving performance, energy efficiency, and latency for specific workloads such as machine learning. Field-programmable gate arrays (FPGAs) have emerged as another attractive platform for accelerating ML workloads, allowing for closer to ASIC performance but allowing for reconfigurability.

Many existing FPGA-based neural network accelerators rely on multiply–accumulate (MAC) operations to implement convolutional or fully connected layers. While these approaches can achieve high performance, they often require specialized datapaths, large memory bandwidth, and complex hardware pipelines. Furthermore, deploying updated models requires regenerating hardware designs through synthesis, place-and-route, and bitstream generation, which can significantly slow development and experimentation cycles.

An alternative approach is the use of lookup-table (LUT) neural networks, where neurons are represented as Boolean functions implemented using small truth tables. In this method, neural computations are replaced with logic operations that map directly to the LUT primitives available in modern FPGA fabrics. LUT-based neural networks offer several advantages, including low-latency inference, efficient hardware utilization, and the ability to represent complex logic functions through the composition of small LUT units. [1]

Recent research has demonstrated that LUT-based architectures can achieve competitive performance for certain tasks while significantly reducing computational overhead. [1] However, many existing implementations still rely on hard-coded LUT values that require full hardware recompilation, thus causing these models to still suffer from the same issue that other FPGA-based neural network implementations have to deal with.

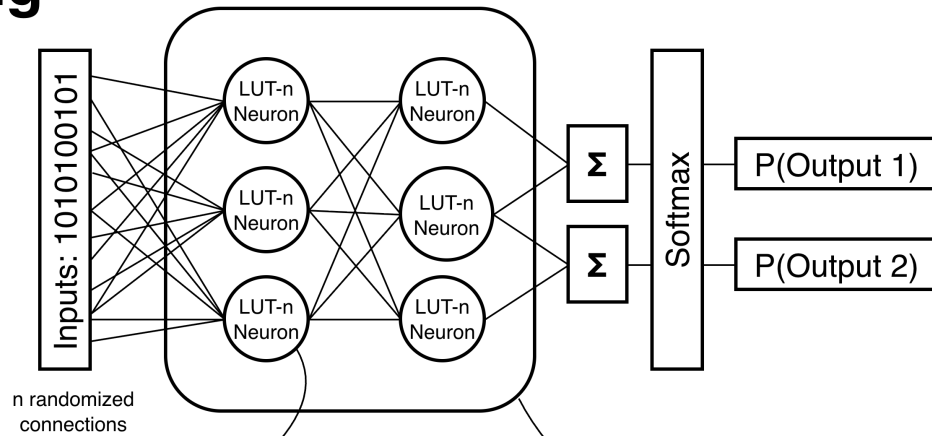
In this work, we explore the design of a reconfigurable LUT neural network architecture that enables dynamic updates to LUT-based models without requiring repeated hardware synthesis. By leveraging configurable LUT primitives and runtime parameter updates, the proposed approach aims to bridge the gap between efficient FPGA inference and flexible ML deployment. To motivate this design, we first review the prior work, the LUT-Based Logic Neural Network (LLNN) architecture, which forms the foundation for the proposed approach. [2]

II. BACKGROUND

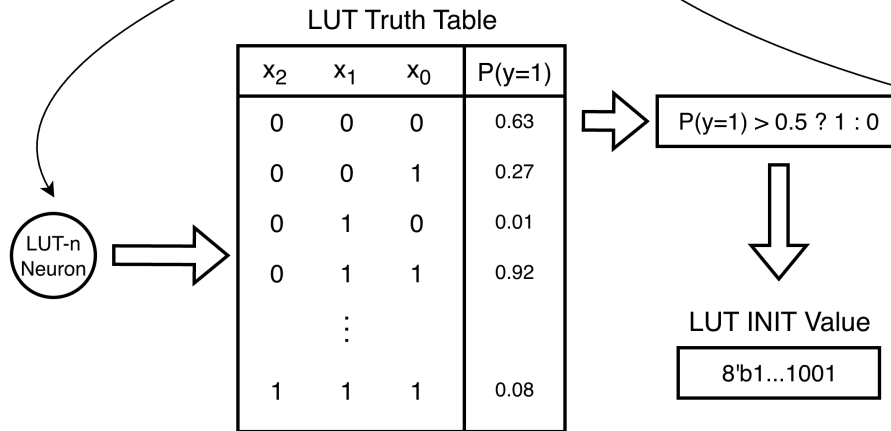
In LLNN, each neuron is implemented as a small k -input lookup table whose output is determined by the truth table associated with the neuron. Instead of performing weighted sums followed by nonlinear activation functions, LLNN neurons compute outputs by selecting values from the lookup table based on their binary inputs. Models are constructed by composing multiple layers of such LUT neurons, where each neuron receives inputs from a subset of neurons in the previous layer.

Training LUT-based neural networks presents a challenge because lookup-table operations are inherently discrete and therefore not directly differentiable. To address this issue, LLNN introduces a differentiable training formulation in which LUT entries are represented as continuous parameters

Training



Hardening



Inference

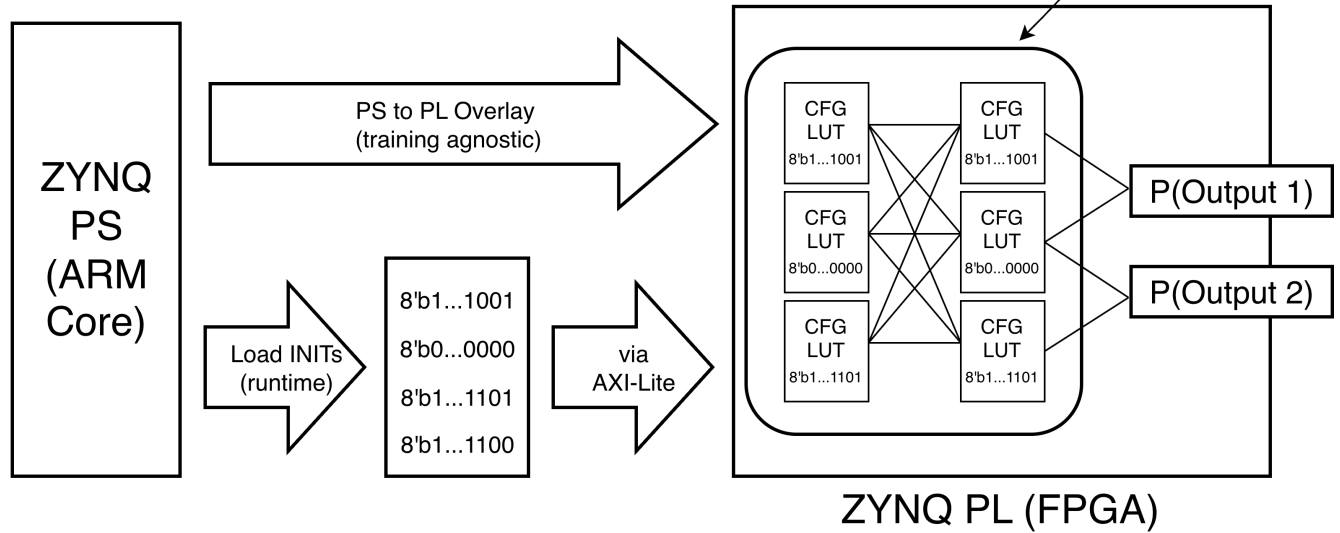


Fig. 1. LiveLLNN Architecture Overview

during training. A probabilistic representation is used to approximate the discrete truth table behavior, allowing gradients to propagate through the network during backpropagation, which uses $2^{(inputs)}$ depending on the number of inputs the LUTs on an FPGA support. In addition, the connectivity between neurons is typically established through randomized input selection. Each LUT neuron receives inputs from a randomly chosen subset of LUTs from the previous layer, as LUTs within FPGA fabrics only have so many possible inputs, but also, depending on the FPGA, can have a range of inputs of 2-6. This randomized connectivity maintains a sparse and hardware-efficient network structure.

Once training is complete, the learned parameters are converted into binary truth table values through a hardening process that maps the continuous representation to discrete LUT entries by applying a threshold ($w_i > 0.5 = 1$, else 0). The hardened model can then be translated into a hardware description language (HDL) representation, where each neuron corresponds to a fixed FPGA LUT primitive, for which LLNN only supports VHDL. As a result, the trained network becomes a purely combinational logic circuit capable of extremely low-latency inference. However, because LUT values are typically embedded directly into the generated hardware design, updating model parameters requires regenerating the FPGA bitstream.

Building upon this framework, we introduce LiveLLNN, a reconfigurable extension of the LLNN architecture that enables runtime updates of LUT parameters without requiring hardware recompilation. The following section describes the design and implementation of the proposed LiveLLNN architecture.

III. METHODOLOGY

The core design philosophy of LiveLLNN is to move beyond the fixed physical mapping of traditional logic neural networks by introducing a flexible, heterogeneous framework. Figure 1 visualizes the architecture detailed in this section. By redefining the neuron as a mutable component rather than a fixed one, we provide an architecture well-suited for edge devices to adapt to concept drift or environmental changes in real-time.

A. Architectural Overview

Our architecture centers around the SoftLUT Neuron, which serves as the fundamental unit within the inference network. Unlike the static LUT INIT values in the original LLNN implementation, where truth tables are permanently hardened into the FPGA bitstream during the physical implementation phase, our SoftLUT approach treats the Boolean logic of each neuron as a runtime variable.

This design leverages the Xilinx CFGLUT5 primitive to implement a serial shift interface through a shift-register logic (SRL) primitive for runtime updates, allowing a 32-bit truth table to be updated without altering the underlying hardware interconnects.

Each neuron implements a Boolean function:

$$y = LUT(x_1, x_2, \dots, x_k)$$

where LUT is a 2^k truth table stored in a CFGLUT5 primitive.

In the architecture pioneered by Ramírez et al. [2], the training process concludes with gate hardening, producing HDL files in which the probabilistic logic is converted into fixed combinational case statements and VHDL components that implement popcount and argmax. Our architecture introduces structural persistence, in which the interlayer connections and the overarching network Directed Acyclic Graph (DAG) can be saved in a `topo.json` file and applied to a new training run, decoupling the wiring topology from the current training run. This allows for rapid iteration since researchers can freeze the hardware-intensive structural topology once and continue to retrain the network for different datasets (e.g., shifting from standard MNIST to rotated MNIST) while ensuring the model remains compatible with a single, existing bitstream.

The primary hardware innovation here is the SoftLUT5 neuron, which leverages the heterogeneous nature of the Zynq SoC by partitioning tasks between the high-level control of the Processing System (PS) and the low-latency execution of the Programmable Logic (PL). Each SoftLUT neuron is a shell that contains a 32-bit initialization shift register and the LUT itself, creating a fabric where the physical wiring is permanently fixed but the Boolean function of every single neuron remains fully reconfigurable at runtime. The Processing System (PS) then communicates with the Programmable Logic (PL) via a memory-mapped AXI-Lite interface, utilizing the `axi_lut_ctrl` module to provide a 64KB addressable range where writing to a specific `gate_id` triggers a 32-cycle serial shift state machine to program the target neuron. The MMIO address specification is shown in Table I; note that the address space is parameterizable and can dynamically grow or shrink to fit the quantity of neurons in the network.

TABLE I
AXI CONFIGURATION CONTROLLER MEMORY MAP

Address Range	Name	Description
0x0000--0x7FFF	GATE_PROG	Write 32-bit truth table
0x8000	STATUS	Read <code>cfg_busy</code> bit (1=busy, 0=ready)
0x8004	TOTAL_GATES	Read-only count of instantiated Soft-LUTs
0x9000--0x9004	NET_I	Inference input
0x9100	NET_O	Inference output

Because the wiring is already established by the bitstream (which can now stay constant between models with the same layer shape), the system bypasses the lengthy (~20 minutes to 2 hours) Vivado synthesis pipeline entirely, enabling model hot-swapping through lightweight PS MMIO transactions that complete in a tiny fraction of a second.

B. Functional Workflows

Our work required us to first verify that the existing LLNN repository would adapt well to PYNQ. As such, we developed two flows: a static LUT flow reflecting the original LLNN implementation but with synchronous clocking for verification and our reconfigurable LUT flow, the focus of this paper.

1) *Static LUT Flow*: In this flow, we primarily used the existing LLNN implementation: truth tables are “baked” into the physical hardware using `case` statements that synthesize into hardened LUTs. Several improvements were made, however, to prepare for integration with modern SoC-FPGA toolchains and the PYNQ framework. First, we added support for SystemVerilog, moving away from the original VHDL implementation to improve modularity and make our implementation more accessible to a wider audience. We also added a global clock and reset infrastructure to the overlay; while the original LLNN was purely combinational, our implementation introduces a sequential data-handling pipeline to ensure stable timing during AXI-Lite register transactions and to facilitate easier timing closure in complex layouts. Furthermore, we integrated a Vivado block design wrapper and automated TCL scripting to streamline the synthesis and implementation of the static bitstream for the PYNQ-Z2 board.

2) *LiveLLNN Reconfigurable Flow*: The reconfigurable flow makes up the majority of our novel contributions to the LLNN field. We introduce a two-phase workflow that separates structural synthesis from parameter optimization. The first time you train a model for a specific network shape, a base model is trained in PyTorch, and its specific randomly generated connectivity graph is exported as a `topo.json` wiring file. This file is then used to generate a structural SystemVerilog netlist composed of `SoftLUT5` shells rather than hardened gates. Once this shell is synthesized into a bitstream, the flow enters a second, zero-synthesis phase where different models can be deployed to the same hardware. By loading the `topo.json` file into a training session with the `--wiring` flag, the system generates new truth tables that reflect the existing physical interlayer connections. These weights are extracted as JSON files and loaded into the `SoftLUT5s` using Python notebook cells via memory-mapped IO on the PS. As demonstrated in our implementation, the PS iterates through these weights, writing truth tables to the AXI address space, as defined in Table I.

C. System Integration and Automation

To ensure that our results can be reproduced and are portable, we developed several automated processes not included in [2] that abstract away the complexity of the heterogeneous toolchain.

- *Automated build system*: We implemented a master Makefile that manages the entire flow of the project, from the initial PyTorch training to the execution of Vivado TCL scripts for bitstream generation. This system allows users to switch between the static and reconfigurable flows with a single command-line argument.

- *High-level Python interface*: By leveraging the PYNQ framework, we provide Jupyter Notebooks that allow users to interact with the FPGA fabric as if it were a standard Python library. This high-level abstraction handles the low-level AXI-Lite memory mapping, making LiveLLNN accessible to machine learning researchers who may not have extensive experience in HDLs.

IV. RESULTS

A. Model Performance

Table II summarizes the performance of the evaluated architectures across several datasets. The LUT-based LLNN architecture achieves high accuracy on MNIST (97%) while maintaining extremely low inference latency of approximately 2 ns per layer. In comparison, a traditional neural network implemented using *hls4ml* achieves comparable accuracy (95%) but requires significantly higher latency (190 ns per layer).

LiveLLNN maintains similarly low hardware latency (2.43 ns) while enabling rapid retraining times. Across datasets, LiveLLNN achieves strong performance on MNIST and rotated MNIST variants while also maintaining reasonable performance on the more complex Fashion-MNIST dataset.

TABLE II
COMPARATIVE PERFORMANCE AND ACCURACY RESULTS

Arch.	Dataset	Accuracy	Latency [†]	Training [‡]
LLNN	MNIST	97%	2.00	36.0
MLP*	MNIST	95%	190.0	3.0
LiveLLNN	MNIST	92.2%	2.43	1.56
LiveLLNN	MNIST Rotated	93.7%	2.43	1.67
LiveLLNN	Fashion	67.7%	2.43	2.37

[†] Latency per layer in ns. [‡] Training time in minutes.

* Model parameters chosen in [2] for comparable accuracy.

B. Latency Analysis

To better understand system-level performance, we measured the runtime of 1000 inference executions on the deployed PYNQ system. The full execution required 3.935 seconds, corresponding to a throughput of 254.1 samples/sec and an average end-to-end latency of 3.93 ms per sample.

Figure 2 shows the breakdown of the latency across the different components of the inference pipeline. The latency distribution reveals that the vast majority of the execution time is spent in software preprocessing and data transfer. Python string parsing accounts for the largest portion of the runtime, followed by AXI communication between the processor system (PS) and programmable logic (PL). In contrast, the actual neural network computation on the FPGA requires only a single clock cycle.

C. Hardware Execution

The neural network accelerator executes entirely within the FPGA programmable logic and requires only one clock cycle at 10 MHz. This corresponds to a hardware execution time of approximately 36 ns per inference (in the theoretical case, i.e. removing all slack), as detailed in Table III.

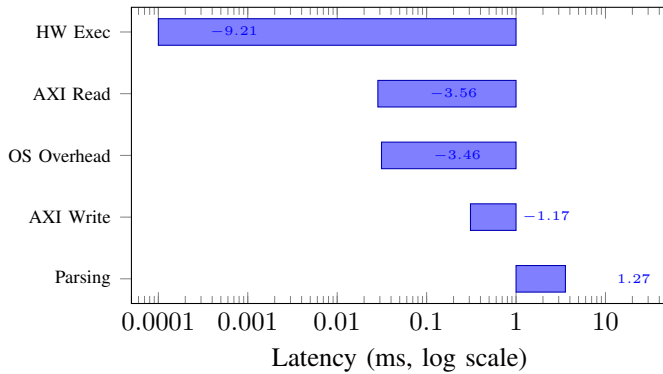


Fig. 2. System Latency Breakdown. The microsecond-scale communication overhead clearly dominates the nanosecond-scale hardware execution logic.

TABLE III
GRANULAR HARDWARE EXECUTION BREAKDOWN (10MHZ CLOCK)

Sub-Component	Latency (ns)	% of Clock Cycle
NN Logic Layers	5	5%
Aggregation Tree	31	31%
Clock Wait / Slack	64	64%
Total PL Execution	100	100%

*Logic propagation time within the single-cycle window.

These results demonstrate that the neural network computation itself is effectively instantaneous compared to the surrounding software overhead. The majority of the measured latency, therefore, originates from preprocessing, system communication, and operating system overhead rather than from the FPGA accelerator.

D. Discussion

Overall, these results demonstrate the primary advantage of LiveLLNN: the accuracy of traditional neural networks and low latency of LUT-based inference are maintained, while allowing for rapid reconfigurability for networks on hardware-constrained edge devices. Namely, the LiveLLNN architecture achieves a latency reduction of nearly two orders of magnitude over MLP using hls4ml while maintaining comparable classification accuracy on MNIST and rotated MNIST, which typically requires a larger network size.

The latency analysis also highlights an important system-level insight: the FPGA accelerator itself contributes negligibly to the total inference time. Instead, the majority of the runtime is dominated by software preprocessing and communication overhead between the processor system and programmable logic.

V. CONCLUSION AND FUTURE WORK

This work presented LiveLLNN, a reconfigurable FPGA framework for deploying lookup-table-based logical neural networks while allowing neuron logic to be updated at runtime. Traditional FPGA workflows require full synthesis and bitstream generation whenever model parameters change, making rapid experimentation slow and impractical. LiveLLNN

addresses this limitation by separating the network topology from the learned parameters, enabling neuron truth tables to be modified dynamically without recompiling the hardware.

The architecture leverages the heterogeneous ZYNQ platform, where the Processing System manages configuration and control while the Programmable Logic performs inference using FPGA LUT primitives. Each neuron is implemented as a SoftLUT5 unit based on the Xilinx CFGLUT5 primitive, allowing its truth table to be updated through an AXI-Lite interface while keeping the FPGA bitstream. This design significantly reduces the time required to deploy new models compared to traditional FPGA compilation workflows.

An end-to-end tool flow was also developed to connect PyTorch training with FPGA deployment. Models can be trained in software, converted into SystemVerilog, synthesized using Vivado, and configured from Python notebook cells via memory-mapped IO on the PS. This workflow allows machine learning experimentation while still taking advantage of the low-latency inference capabilities of FPGA hardware.

Future work will focus on scaling the architecture to support larger logical neural networks, improving automation in the training to hardware pipeline, and exploring runtime adaptation, where neuron truth tables could be updated dynamically as new data is observed. These directions could enable LUT-based neural networks to support more complex tasks and make them increasingly useful for edge computing environments where efficiency and adaptability are critical.

VI. WORK BREAKDOWN

Raphael Huang: Reconfigurable overlay HDL implementation, architecture design, and initial PYNQ board setup.

Chad Baker: Implemented Windows/WSL support, developed and debugged the Vivado build flow, PYNQ board setup, and validated full project workflow.

Marco Frank: Linux support, master Makefile orchestration, adding System Verilog support to LLNN, automated Vivado building using TCL scripting, debugging Vivado build flow, GitHub management.

ACKNOWLEDGMENT

We would like to thank Professor Jason Eshraghian at the University of California, Santa Cruz, for supporting our project within the ECE 110/210: Brain-Inspired Machine Learning course.

APPENDIX

The complete source code, including installation instructions, Docker setup (for training and generating the HDL), and the Makefile workflow, is available at: <https://github.com/MrPoloGit/LiveLLNN>

To replicate the results:

- 1) Clone the repository.
- 2) Install Vivado (if on Linux, set up the path and uncomment the Linux portion of the Makefile).
- 3) Follow the steps in the README.md.

REFERENCES

- [1] Z. Guo, “A survey on lut-based deep neural networks implemented in fpgas,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.07367>
- [2] I. Ramírez, F. J. García-Espinosa, D. Concha, L. A. Aranda, and E. Schiavi, “Llnn: A scalable lut-based logic neural network architecture for fpgas,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 73, no. 1, pp. 347–359, 2026.